

PRAMANA — Everything, In Plain Language

A briefing for the whole team. No engineering background assumed.

By the end of this document you should be able to explain what we are building, why it is better than what everyone else will build, and answer a judge's question without looking at anyone else for help.

Read it twice. The second time, read it out loud.

Part 1 — What are we even talking about?

The aircraft

India flies large surveillance drones — the kind that stay in the air for **eighteen hours at a time**, at altitudes up to 28 000 feet. DRDO's own platform of this type is called **TAPAS BH-201**. In the jargon these are called **MALE UAVs**: Medium Altitude, Long Endurance, Unmanned Aerial Vehicles.

These are not quadcopters. They are aeroplane-sized, they carry two engines, and they fly for most of a day at a time, usually going round and round in a slow racetrack pattern over an area of interest.

The engine

They are powered by **piston engines** — the same basic kind of engine as in a car, with cylinders, pistons and a crankshaft, but built for aviation. DRDO built one for exactly this purpose: a **180 horsepower aero-diesel**, developed at their Vehicles Research and Development Establishment. It has run 1100+ hours on test and has been trialled high in the mountains at Leh and Changla.

The engine is also **turbocharged**, which matters a lot to our story — more on that shortly.

The problem

Nobody is on board. If the engine starts to fail, there is no pilot to notice a strange noise, no engineer to look at the gauges and think "that's odd." All anyone on the ground has is a stream of numbers coming down a radio link.

So the question DRDO is asking is: **can software watch those numbers and work out what is happening inside an engine it cannot see, touch or hear?**

Part 2 — What is wrong with how it is done today

Today's systems are **threshold-based**. That means: every sensor has a red line, and if a number crosses it, an alarm goes off.

Oil pressure below 2 bar? Alarm. Cylinder head above 135 °C? Alarm.

That is it. That is the whole system.

DRDO says so themselves, in the problem statement, in their own words:

"Conventional engine monitoring systems used in UAVs are primarily threshold-based and reactive in nature."

That sentence is the most useful thing they wrote, and most teams will skip straight past it. It is DRDO telling us precisely what to beat.

Why red lines fail — three reasons, and you should know all three

1. They are late by definition.

A red line is placed at the point where *damage begins*. So an alarm is, by definition, the announcement that damage **has already begun**. It is not a warning. It is a notification of something that already happened.

A parameter drifting steadily toward its limit carries the same information *hours earlier* — but only if you know what it *should* be reading. A fixed number does not know that.

2. They cannot understand context.

A cylinder head temperature of 128 °C is **completely fine** at full power on a hot day near the ground. The same 128 °C is a **serious problem** at cruise power at 20 000 feet, where the air is freezing and the engine is working gently.

A single number cannot tell those two situations apart. So engineers set the limit for the worst case — and the system loses sensitivity everywhere else.

3. They cannot doubt their own sensors.

This is the big one. To a threshold system, "*the engine is overheating*" and "*the thermometer is broken*" look **exactly the same**. Both are just a number that went too high.

On a manned aircraft, a pilot cross-checks: is the oil temperature up too? Is it making a noise? Does it feel different? On an unmanned aircraft **there is nobody to do that**. So a forty-dollar failed sensor can abort an eighteen-hour mission.

Part 3 — What a digital twin actually is

The phrase gets used loosely, so here is the honest version.

A threshold system compares a sensor to a red line. A digital twin compares a sensor to what physics says it *should* read, right now, at this exact operating point.

The gap between those two numbers is the entire product.

We build a working mathematical model of the engine — a simulation that knows how air flows in, how fuel burns, how heat moves, how the turbocharger spins. We run it **live, alongside the real engine**, fed with the same throttle setting, the same altitude, the same air temperature.

Then, every second, we ask: *what does my model say cylinder 2's exhaust temperature should be?* And we compare that to what the sensor actually reads.

The difference is called a residual. Residuals are the heart of everything we build.

In a healthy engine, every residual sits at **zero**. When something starts to go wrong, one or more of them starts to drift — and it drifts *long before* anything reaches a red line.

Why this is such a big deal

A red line is a fixed number. A twin's prediction is a **moving target that already accounts for altitude, temperature, power setting and airspeed**.

So the twin can spot a problem that is nowhere near any limit — because it knows what "normal for right now" looks like, and *this isn't it*.

Part 4 — Our actual idea, and why it is better

Now the part that separates us from the other teams.

Most teams that build a twin will stop at the previous section: model the engine, compare to sensors, raise a flag. That is already better than a threshold system.

We go one step further, and that step is the whole project.

The insight: an engine tells you the same thing several different ways

Here is the key fact. An engine has **more sensors than it has independent things happening**. That sounds like waste. It is actually the most valuable property the system has.

Take **air mass flow** — how much air is going into the engine every second. It is one of the most useful numbers you can know about an engine, and on most aircraft **it is not measured at all**. There is no sensor for it.

But you can *work it out* — **four different ways**, from sensors that are already there:

WAY	HOW
1	From the manifold pressure, air temperature and engine speed — basic gas physics
2	From the turbocharger's speed and how hard it is compressing — read off the compressor's performance chart
3	From how much fuel is being injected and how much oxygen is left in the exhaust
4	From the pressure drop across a restriction in the intake pipe

Four completely different routes. **Different sensors, different physics, no shared failure.**

In a healthy engine with honest sensors, all four give the same answer.

And here is the whole thesis

Health is not how far a measurement is from a limit. Health is how much the engine's several independent answers agree with each other.

When something breaks, they stop agreeing. And here is the beautiful part:

Different faults break the agreement in different, predictable patterns — and the pattern tells you what the fault is.

We do not have to teach the computer what each fault looks like. **The pattern falls out of the physics**, because we know which sensors and which equations went into each route.

This is called **over-determination**, and it is why our project is called **PRAMANA** — a Sanskrit word meaning *the means by which a thing is known to be true*. It is the classical Indian term for how you establish that something is true — and specifically, for how independent sources of knowledge corroborate one another. That is literally

what our system does.

An analogy that works on anyone

Imagine four witnesses to the same event, questioned separately.

- All four tell the same story → you believe it.
- One tells a different story → **that witness is the problem.**
- Three tell a story and one is confused about a specific detail → you can work out which one is confused, **and about what.**

You do not need to have met the liar before to catch them. **You just need enough independent accounts that the inconsistency has nowhere to hide.**

That is our system. The engine is the event; the sensors are the witnesses.

Part 5 — The thing nobody else will do

Everything in this section falls out of the idea above **for free**. That is why our architecture stays small: these are not extra features we bolted on, they are consequences.

Consequence 1 — We can tell a broken engine from a broken sensor

This is our headline. Remember this one above all others.

Think about what happens in each case:

- **A real engine problem** disturbs *several* related measurements at once, in a physically connected way. A clogging injector on cylinder 2 makes that cylinder run hot, changes the fuel flow, upsets the energy balance, and puts a wobble in the crankshaft rotation. **Five things move together, and they move in a way that makes physical sense.**
- **A broken sensor** disturbs exactly **one** number — the one it is reporting. Nothing else changes, because nothing else *is* changing. The engine is completely fine.

So we look at the *pattern*, not the *value*. **One number moving alone, with nothing corroborating it, is a sensor. Several numbers moving together in a connected way is an engine.**

There is an even sharper version. If the manifold pressure sensor starts reading high, it corrupts route 1 **upward** and route 2 **downward at the same time** — because that same pressure enters one calculation as a density and the other as a ratio. **No real engine fault can imitate that particular double-signature.** It is a fingerprint that only a lying sensor leaves.

Why this wins: the problem statement explicitly lists sensor drift as a fault to detect, right alongside real engine faults. DRDO put it there on purpose. Almost nobody will notice, and almost nobody will separate the two.

Consequence 2 — Sensors we do not have

If a quantity can be worked out four ways, then **any one of those ways can supply it when there is no sensor at all.**

So we can report things the aircraft physically cannot measure: peak pressure inside each cylinder, how close it is to damaging knock, turbocharger shaft speed, how efficiently each individual cylinder is burning fuel.

This is called **virtual sensing**, and DRDO asks for it by name.

Consequence 3 — We can prove we are right without any real failure data

This is the hardest question a judge can ask, and we have a real answer.

The problem: to prove a system predicts engine failures, you would normally need recordings of engines actually failing. **That data does not exist** for these engines. Nobody is going to run a fleet of aero engines to destruction to create a training set — it would take years and cost a fortune.

This is precisely why DRDO has issued a request for proposals in this area rather than simply buying a product.

Our answer: **the system validates against itself**. If four independent routes to the same number agree across the entire flight envelope — sea level to 28 000 feet, cold and hot, low power and high — then our model has got the physics right in the quantities that matter. **No external reference was needed to establish that.**

That is a genuine, defensible answer to the hardest question in this field.

Consequence 4 — It moves to a different engine cheaply

We express the engine's health in **ratios and proportions** rather than absolute readings — efficiency, fraction of load, and so on. Expressed that way, the relationships belong to the *class* of engine rather than to the one specific machine.

So a model built on DRDO's engine largely works on a different one. **Honestly stated:** it needs about ten healthy readings from the new engine to correct a small offset — not a full run-to-failure campaign. That is a weaker claim than "it transfers perfectly," and a far stronger one than anything a system trained on raw numbers can offer.

Consequence 5 — The second engine is a free experiment

The aircraft has **two engines**. Two nominally identical machines, built to the same spec, drawing from the same fuel tanks, flying through the same air, on the same mission, for eighteen hours.

That is the best controlled experiment available in aviation, and it costs nothing to use.

Subtract one engine's readings from the other. Everything shared — air temperature, altitude, fuel batch quality, weather — **cancels out exactly**. What is left is specific to one engine, which means it is either degradation or instrumentation.

The rule, which is how a maintenance officer already thinks:

Both engines drifting together is the environment or the fuel. One engine drifting alone is that engine.

Every team that models a single engine throws this away completely.

Consequence 6 — Going round in circles is a gift

An endurance mission is not a varied flight. It is **hours of flying the same racetrack pattern** at the same altitude and the same power setting.

Every lap returns the engine to almost exactly the same operating condition as the last lap. That means we can compare like with like, directly, without having to account for anything changing.

An 18-hour mission with a 20-minute orbit gives us roughly **54 perfectly matched samples per flight** — free, just by noticing what the aircraft is already doing.

Part 6 — The part with "AI" in it

The problem statement asks for AI/ML. Here is what we actually use it for, and **why ours works when most teams' will not**.

The mistake everyone else will make

The obvious approach: feed all the raw sensor readings into a neural network and let it learn what "normal" looks like.

This fails, and it fails in a specific, predictable way. The network learns the *flight profile*, not the engine. Then every time the throttle moves or the aircraft climbs, all the numbers change — and the network screams that something is wrong.

Constant false alarms. This is the single biggest reason naive builds fail, and a lot of teams will discover it on the last day.

What we do instead

Our AI never looks at raw sensor readings. It looks at the residuals — the gaps between measurement and prediction.

Because the residuals are **zero when healthy, no matter what the aircraft is doing**, the network has an easy job. It learns one simple thing: *healthy means near zero*. Climbing, descending, throttling up — the residuals stay at zero, so the network stays quiet.

The physics does the hard work of removing the flight conditions. The AI only has to notice that something is off. That is why ours is stable and theirs will not be.

We can even *prove* this: same network, trained on raw signals versus trained on residuals, side by side. One bar chart, one afternoon of work, and our central claim becomes a measured result instead of an assertion.

The three models — that is all

1. **A predictor** — physics does the heavy lifting; a small network learns only the leftover error. Because it only learns a small correction, it needs very little data and **cannot produce nonsense at an altitude it has never seen**.
2. **An anomaly detector** — trained only on healthy data, it flags "this doesn't look like a healthy engine" without being told what the fault is.
3. **A classifier plus a life estimator** — names the fault, and estimates how long the component has left.

Remaining useful life — and the honest way to report it

We estimate it **two independent ways**: once by integrating the physical damage forward, once with a neural network. We report both, and we advise on the more cautious one.

"Physics says 4.1 hours, the network says 3.7. We advise on 3.7."

And we **always** show a range, never a single number: 2.4 to 4.1 hours.

A bare number reads as overconfident to anyone who has worked with real hardware. A range reads as someone who has thought about it. Volunteer the uncertainty before anyone asks.

Part 7 — When the system is not sure, it runs an experiment

This is the most advanced thing we do, and it is genuinely unusual.

Sometimes two possibilities look identical from the outside. The clearest case: *is cylinder 2's injector clogging, or is cylinder 2's temperature sensor drifting?* Early on, when the fault is small, both look the same — one temperature reading creeping up.

Most systems would guess. **Ours asks the engine a question.**

It commands a tiny, deliberate wiggle in the fuel going to that cylinder — up a little, down a little — and watches **how strongly the temperature responds.**

- If the **injector is clogged**, it can only deliver part of what we asked for, so the temperature wiggles **weakly**.
- If the **injector is fine and the sensor is drifting**, the temperature wiggles at **full strength** — the sensor is adding a constant offset, and *a constant offset does not affect a wiggle at all*.

We are not looking at the level. We are looking at the response. The sensor's error cancels out completely, because a constant cannot change how much something moves.

The test takes a few seconds and stops as soon as it is confident.

Is it safe? Yes, and we have a specific answer

- The wiggle is **within the range the engine's own control unit already uses** in normal operation ($\pm 3\text{--}5\%$)
- It lasts **a few seconds**
- It only runs when the system is **genuinely uncertain** — never routinely
- It is **switched off** during takeoff, climb, approach, or if one engine is already out
- It **stops immediately** if anything approaches a limit
- On a two-engine aircraft, **one engine at a time**

This changes what kind of system we are. A monitor observes and infers. PRAMANA, when it knows that watching is not enough, **designs and runs an experiment.**

Part 8 — We do not just report. We decide.

Look at the title of the problem statement again:

"...Health Monitoring, Fault Prediction and *Mission Reliability Enhancement*..."

Enhancement. Not monitoring. Not reporting. *Enhancement.*

Almost every team will read straight past that word. They will show a fault alert and stop. Detection is table stakes.

Because we have a working model of the engine *and* we know the route still to fly, we can **run the whole rest of the mission forward in fast-forward**, hundreds of times, with the damage we have detected — and count how many of those runs finish safely.

That gives a number no other team will have:

Continue the mission: 58% chance of completing it. Reduce power to 78%: 94%. Turn back now: 99%.

Three options, three probabilities. **Not a red light — a decision.**

And crucially, we also show **what the safe option costs**:

"Derating costs you forty minutes on station."

Reliability advice that ignores the mission is advice that gets ignored. Showing the trade-off is what makes this read as a defence product rather than a maintenance tool.

We also compute the **point of no return** — the last moment the aircraft can still get home — using the *actual degraded* fuel consumption, not the figure in the book. It sits on the map as a moving marker.

The moment to plan the demo around

Hand the mouse to a judge. Let them drag the cruise altitude down and watch the probability change in real time.

A judge changing the mission themselves is worth more than any amount of model architecture.

Part 9 — Why the system is split in two

The problem statement asks for both **edge computing** (on the aircraft) and **cloud analytics** (on the ground). Most teams will treat that as an architecture preference. **It is arithmetic, and saying so is far more persuasive.**

Vibration signals that reveal knocking and worn bearings need to be sampled **five to twenty thousand times per second**. The radio link from the aircraft to the ground carries a few tens of kilobits per second — **and it is shared with the video feed and the flight commands.**

Raw vibration data cannot go down that link. Not in our design, not in anyone's design, not ever.

Therefore the fast processing **has to** happen on the aircraft, and only the conclusions travel down. The split is not a choice; it is forced by physics and radio bandwidth.

Three lanes of data:

LANE	SPEED	WHERE IT LIVES	SENT TO GROUND?
Fast	5 000–20 000 /sec	On the aircraft only	No — only the conclusions
Slow	10–50 /sec	Aircraft → ground	Yes
Health	1 /sec	Computed on the aircraft	Yes — a few hundred bytes

The deployment story, in one sentence

Only the bottom two layers know the data is simulated. **Swap the simulator for a real engine and a real radio, and nothing above them changes at all.**

That single sentence is our answer to "*how does this get onto a real aircraft?*", and it is worth a slide of its own.

We also send our data as genuine **CAN bus** messages — the actual electrical standard real engine controllers use. The problem statement names CAN specifically. Having the raw CAN traffic scrolling in the corner of the screen during the demo says *this speaks the same language as a real engine* more convincingly than any slide could.

Part 10 — "But your data is fake"

Someone will ask this. Probably early. Here is the answer, and it is a strong one.

"It's synthetic — and so is C-MAPSS, the dataset this entire field has been benchmarked on for eighteen years, published by NASA. Our red lines come from the engine's actual type certificate. Our fault signatures come from published failure-mode literature. Our wear model is Archard's law and Miner's rule — established engineering, not a random number generator. And the pipeline doesn't care where the data comes from: here it is, running unmodified, on NASA's flight-condition dataset, with the score."

Then show it. That last sentence is worth building for, and one of us is doing exactly that on Day 1.

There is a second half to the answer. Because our simulation *contains* the wear as an actual quantity we control, **every single training example comes with a perfect answer attached** — we know exactly how much life was left, because we defined it. Real-world data almost never has that. **NASA built C-MAPSS for precisely this reason.**

Part 11 — The part that makes us look honest

We have a slide listing our own limitations. This is deliberate, and it is not modesty.

We will say plainly:

- None of our individual techniques are new. All of them are established methods with substantial literature. **What we claim is the coherent assembly**, plus a real answer to the validation problem this domain has never had a good answer for.
- We did **not** implement the strictest form of physics-informed neural network — and we explain why: high risk, little benefit for this kind of system, and if it failed to converge we would have nothing.
- **Three pairs of faults cannot be told apart** from steady measurements alone. We name them. That limitation is exactly *why* we built the experiment-running capability.
- Cross-engine transfer is **approximate** and needs a small correction.
- We still compare residuals to a threshold — over-determination changes *what* gets thresholded, not the existence of a threshold.

Why volunteer all that?

An evaluator who catches a limitation you hid will distrust everything else you said. An evaluator who finds every limitation already identified and bounded reads the rest of your claims as reliable.

In a technical evaluation, that trade strongly favours us. **The honesty slide makes us more credible, not less.**

The same principle applies to every number on every slide: if it is measured from our simulator, we say so, in the caption, every time.

Part 12 — What the demo looks like

Four minutes. Rehearsed until it is boring.

TIME	WHAT HAPPENS
0:00	Healthy engine cruising at 18 000 ft. Our prediction sits exactly on the measurement. <i>"Everything is green. A threshold system sees exactly what you see. Watch."</i>
0:40	We start clogging cylinder 2's injector, slowly. Every limit stays green. Our residual lifts within seconds. Cylinder 2 turns amber in the 3D engine.
1:20	<i>"Injector fouling, cylinder 2, 91%."</i> Open the explanation panel: which measurements moved, how they match the known fingerprint. The AI and the physics agree — two independent methods, same answer.
2:00	Remaining life: 3.7 hours, range 2.4–4.1. Both estimation methods shown. We advise on the cautious one.
2:25	The decision: continue 58%, derate 94%, turn back 99%. <i>"Derating costs forty minutes on station."</i> Hand the mouse to a judge.
3:00	The twist. We break a <i>sensor</i> this time, not the engine. The system says: sensor fault — the engine is fine. <i>"A threshold system just aborted an eighteen-hour mission for a forty-dollar thermocouple."</i>
3:30	Scrub the timeline backwards through the whole event. Generate the flight report.
3:50	Close on deployment , not on technology. What flies on the aircraft, what runs on the ground, what changes with a real engine. <i>Nothing above layer 1.</i>

The 3:00 beat is the line they will repeat to the next team. Everything else in the build exists to make that moment land.

Part 13 — Answers you should have ready

Practise these until they come out without thinking. **Whoever is least technical on the team should be able to answer the first three.**

"Your data is synthetic. Why should we believe it?"

Because the mechanism is real even where the numbers are generated. Red lines from the type certificate, fault signatures from published failure literature, wear from Archard and Miner — not a random number generator. C-MAPSS is synthetic too, and this field has benchmarked on it for eighteen years. And our pipeline doesn't care — here it is on NASA's data, with the score.

"What does this give us that an alarm doesn't?"

Fifteen minutes of warning instead of zero, because we compare against what physics says the reading should be rather than a fixed number. Context — 128 °C means different things at sea level and at 20 000 feet. And we can doubt our own sensors, which an alarm can never do.

"How is this a twin and not a dashboard with a model behind it?"

Because the model runs live, in step with the engine, and re-tunes itself as the engine ages. Its internal parameters *are* the health report — in physical units an engineer can argue with. And it runs faster than real time, so we can fast-forward the rest of the mission and give you a completion probability. A dashboard cannot do that.

"Which engine, and does it transfer?"

DRDO's own VRDE 180 hp aero-diesel, using your published altitude-trial figures. It transfers to the same class of engine for about ten healthy readings' worth of correction — not a run-to-failure campaign, which would take years.

"How does this get onto an aircraft?"

Only the bottom two layers know the data is simulated. Replace them with a real engine controller and a real datalink and nothing above changes. We already speak CAN, the bus real controllers use. Next step is recalibration against dynamometer data, then shadow mode on recorded flights.

"What's your false alarm rate?"

[Give the measured number from 100 healthy simulated flights.] And we report it per flight hour, because that is the number a maintenance officer actually cares about.

"Why trust an RUL number from a neural network?"

Don't trust it alone — that's why we compute it twice, once from physics and once from the network, report both, and advise on the more conservative. And we never give a single number without its range.

"What did you not build, and why?"

[Answer directly and specifically — this is the honesty slide, and it is a question we want.]

Part 14 — The five things to remember if you remember nothing else

1. **A threshold compares a sensor to a red line. A twin compares it to what physics says it should read right now.** The gap is the product.
 2. **An engine tells you the same thing several independent ways. Health is how well those answers agree.** Different faults break the agreement in different, predictable patterns.
 3. **A broken engine moves several measurements together in a connected way. A broken sensor moves exactly one, with nothing backing it up.** That is how we tell them apart, and nobody else will.
 4. **Our AI never sees raw sensor readings — only the gaps.** That is why it does not false-alarm every time the throttle moves.
 5. **We do not report. We decide.** Continue 58%, derate 94%, turn back 99% — with the cost of the safe option stated in minutes on station.
-

A note on the name

Pramāṇa (प्रमाण) is the Indian epistemological term for *a valid means of knowledge* — the question of how one establishes that something is true, and how independent sources of knowledge corroborate one another.

We chose it because it **describes the system's actual mechanism rather than decorating it**. Our system establishes engine health precisely by asking whether independent sources of knowledge agree.

It is a good name to say out loud in front of an Indian defence research organisation, and it is a better name because it is accurate.